תכנון דינמי

חזרה קצרה | תרגילים

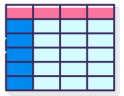
תכנון דינמי

תכנון דינמי היא שיטה המתאימה לבעיות אופטימיזציה אותן ניתן לבטא באמצעות בעיות קטנות יותר מאותו הסוג.

המטרה – לא לחשב פעמיים את אותה בעיה קטנה, אלא לשמור את תוצאות החישוב לשימוש נוסף בהמשך.

דרך הפעולה

ניסוח נוסחה רקורסיבית לתיאור הפתרון



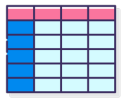
תיאור מבנה נתונים השומר את המידע המחושב בכל איטרציה

מציאת אלגוריתם איטרטיבי "מלמטה למעלה" המשתמש במבנה הנתונים לצורך בניית הפתרון

הבדלים בין תכנון דינמי לשיטות אחרות שלמדנו לבעיות אופטימיזציה

הפרד ומשול – בשיטת "הפרד ומשול" **תתי הבעיות** שפותרת הרקורסיה **זרות**,

לרוב בשיטת תכנון דינמי **תתי הבעיות יהיו חופפות** ונשמור את המידע החופף במבנה נתונים שנגדיר לשימוש עתידי.



הגישה החמדנית – בשתי הגישות נשתמש בתכונת **תת המבנה האופטימלי**.

ההבדל **שבגישה החמדנית לא נשתמש** במידע נוסף שחושב באיטרציות קודמות לצורך מציאת הפתרון האופטימלי **ובתכנון דינמי כן**.

תרגיל חימום - חישוב מספרי פיבונאצ'י

תזכורת – סדרת פיבונאצ'י

סדרת פיבונאצ'י היא סדרה ששני איבריה הראשונים הם 1, 0 וכל איבר לאחר מכן שווה לסכום שני קודמיו.
איברי הסדרה נקראים **"מספרי פיבונאצ'י"**. הנוסחה למציאת האיבר ה-n: $A_n = A_{n-2} + A_{n-1}$

תרגיל חימום - חישוב מספרי פיבונאצ'י

תזכורת – סדרת פיבונאצ'י

סדרת פיבונאצ'י היא סדרה ששני איבריה הראשונים הם 1, 0 וכל איבר לאחר מכן שווה לסכום שני קודמיו.
איברי הסדרה נקראים **"מספרי פיבונאצ'י"**. הנוסחה למציאת האיבר ה-n: $A_n = A_{n-2} + A_{n-1}$

פתרון רקורסיבי נאיבי

Fibonacci(n):
1. **if** n = 1
2. **then return** 0
3. **If** n = 2
4. **then return** 1
5. **return** Fibonacci(n-2) + Fibonacci(n-1)

תרגיל חימום - חישוב מספרי פיבונאצ'י

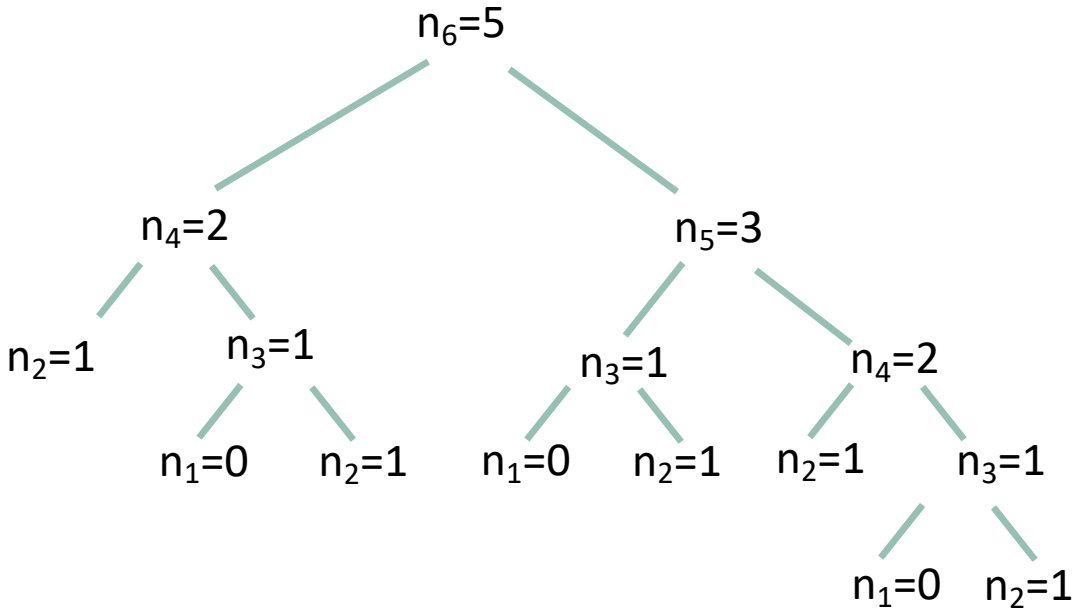
תזכורת – סדרת פיבונאצ'י

סדרת פיבונאצ'י היא סדרה ששני איבריה הראשונים הם 1, 0 וכל איבר לאחר מכן שווה לסכום שני קודמיו.
איברי הסדרה נקראים **"מספרי פיבונאצ'י"**. הנוסחה למציאת האיבר ה-n: $A_n = A_{n-2} + A_{n-1}$

פתרון רקורסיבי נאיבי

Fibonacci(6):

1. **if** n = 1
2. **then return** 0
3. **If** n = 2
4. **then return** 1
5. **return** Fibonacci(n-2) + Fibonacci(n-1)



תרגיל חימום - חישוב מספרי פיבונאצ'י

תזכורת – סדרת פיבונאצ'י

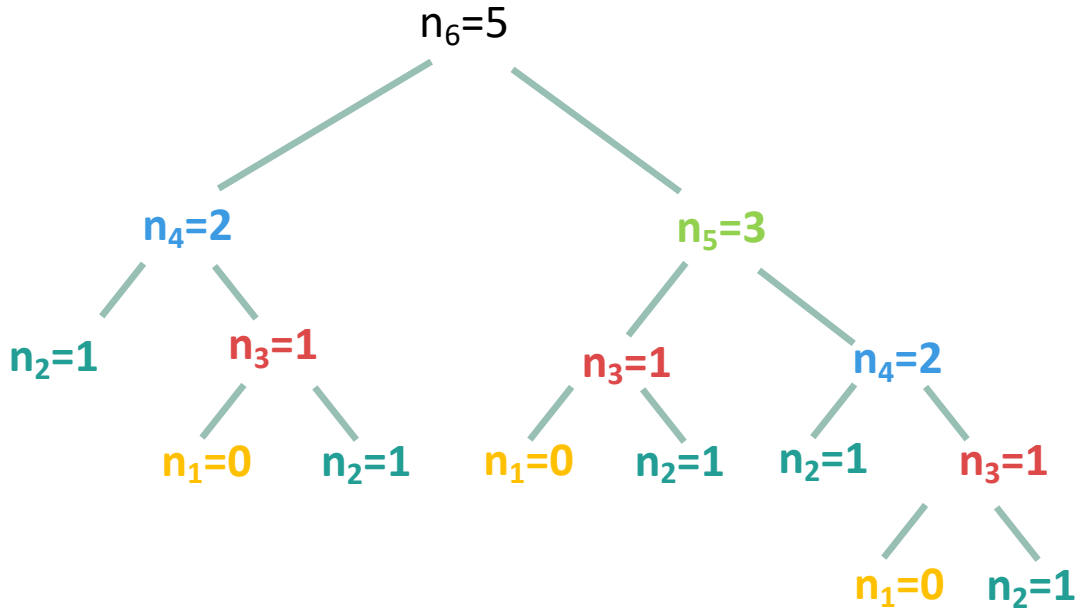
סדרת פיבונאצ'י היא סדרה ששני איבריה הראשונים הם 1, 0 וכל איבר לאחר מכן שווה לסכום שני קודמיו.
איברי הסדרה נקראים **"מספרי פיבונאצ'י"**. הנוסחה למציאת האיבר ה-n: $A_n = A_{n-2} + A_{n-1}$

פתרון רקורסיבי נאיבי

Fibonacci(6):

1. **if** n = 1
2. **then return** 0
3. **If** n = 2
4. **then return** 1
5. **return** Fibonacci(n-2) + Fibonacci(n-1)

זמן ריצה: $O(2^n)$ **עלות מקום**: O(n)



תרגיל חימום - חישוב מספרי פיבונאצ'י

תזכורת – סדרת פיבונאצ'י

סדרת פיבונאצ'י היא סדרה ששני איבריה הראשונים הם 1, 0 וכל איבר לאחר מכן שווה לסכום שני קודמיו.
איברי הסדרה נקראים **"מספרי פיבונאצ'י"**. הנוסחה למציאת האיבר ה-n: $A_n = A_{n-2} + A_{n-1}$

פתרון באמצעות תכנון דינמי

ניסוח נוסחה רקורסיבית לתיאור הפתרון

מציאת אלגוריתם איטרטיבי "מלמטה למעלה"

תיאור מבנה נתונים השומר את המידע המחושב בכל איטרציה

תרגיל חימום - חישוב מספרי פיבונאצ'י

תזכורת – סדרת פיבונאצ'י

סדרת פיבונאצ'י היא סדרה ששני איבריה הראשונים הם 1, 0 וכל איבר לאחר מכן שווה לסכום שני קודמיו.
איברי הסדרה נקראים **"מספרי פיבונאצ'י"**. הנוסחה למציאת האיבר ה-n: $A_n = A_{n-2} + A_{n-1}$

פתרון באמצעות תכנון דינמי

ניסוח נוסחה רקורסיבית לתיאור הפתרון

$$Fibonnaci(n) = \begin{cases} 0 & n = 1 \\ 1 & n = 2 \\ Fibonacci(n-2) + Fibonacci(n-1) & else \end{cases}$$

מציאת אלגוריתם איטרטיבי "מלמטה למעלה"

תיאור מבנה נתונים השומר את המידע המחושב בכל איטרציה

תרגיל חימום - חישוב מספרי פיבונאצ'י

תזכורת – סדרת פיבונאצ'י

סדרת פיבונאצ'י היא סדרה ששני איבריה הראשונים הם 1, 0 וכל איבר לאחר מכן שווה לסכום שני קודמיו.
איברי הסדרה נקראים **"מספרי פיבונאצ'י"**. הנוסחה למציאת האיבר ה-n: $A_n = A_{n-2} + A_{n-1}$

פתרון באמצעות תכנון דינמי

ניסוח נוסחה רקורסיבית לתיאור הפתרון

$$Fibonnaci(n) = \begin{cases} 0 & n = 1 \\ 1 & n = 2 \\ Fibonacci(n-2) + Fibonacci(n-1) & else \end{cases}$$

מציאת אלגוריתם איטרטיבי "מלמטה למעלה"

תיאור מבנה נתונים השומר את המידע המחושב בכל איטרציה

נשתמש בשני משתנים n1, n2 לשמור את שני המספרים האחרונים שחושבו בסדרה

תרגיל חימום - חישוב מספרי פיבונאצ'י

תזכורת – סדרת פיבונאצ'י

סדרת פיבונאצ'י היא סדרה ששני איבריה הראשונים הם 1, 0 וכל איבר לאחר מכן שווה לסכום שני קודמיו.
איברי הסדרה נקראים **"מספרי פיבונאצ'י"**. הנוסחה למציאת האיבר ה-n: $A_n = A_{n-2} + A_{n-1}$

פתרון באמצעות תכנון דינמי

ניסוח נוסחה רקורסיבית לתיאור הפתרון

$$Fibonnaci(n) = \begin{cases} 0 & n = 1 \\ 1 & n = 2 \\ Fibonacci(n-2) + Fibonacci(n-1) & else \end{cases}$$

מציאת אלגוריתם איטרטיבי "מלמטה למעלה"

Fibonacci(n):

```
1.  if n = 1
2.    then return 0
3.  if n = 2
4.    then return 1
5.  n1 ← 0
6.  n2 ← 1
7.  for i ← 3 to n
8.    do temp ← n1 + n2
9.       n1 ← n2
10.      n2 ← temp
11. return n2
```

תיאור מבנה נתונים השומר את המידע המחושב בכל איטרציה

נשתמש בשני משתנים n1, n2 לשמור את שני המספרים האחרונים שחושבו בסדרה

תרגיל חימום - חישוב מספרי פיבונאצ'י

תזכורת – סדרת פיבונאצ'י

סדרת פיבונאצ'י היא סדרה ששני איבריה הראשונים הם 1, 0 וכל איבר לאחר מכן שווה לסכום שני קודמיו.
איברי הסדרה נקראים **"מספרי פיבונאצ'י"**. הנוסחה למציאת האיבר ה-n: $A_n = A_{n-2} + A_{n-1}$

פתרון באמצעות תכנון דינמי

ניסוח נוסחה רקורסיבית לתיאור הפתרון

$$Fibonnaci(n) = \begin{cases} 0 & n = 1 \\ 1 & n = 2 \\ Fibonacci(n-2) + Fibonacci(n-1) & else \end{cases}$$

מציאת אלגוריתם איטרטיבי "מלמטה למעלה"

Fibonacci(n):

```
1.  if n = 1
2.    then return 0
3.  if n = 2
4.    then return 1
5.  n1 ← 0
6.  n2 ← 1
7.  for i ← 3 to n
8.    do temp ← n1 + n2
9.       n1 ← n2
10.      n2 ← temp
11. return n2
```

תיאור מבנה נתונים השומר את המידע המחושב בכל איטרציה

נשתמש בשני משתנים n1, n2 לשמור את שני המספרים האחרונים שחושבו בסדרה

זמן ריצה: O(n) **עלות מקום**: O(1)

תרגיל חימום - חישוב מספרי פיבונאצ'י

אפשר לראות את ההבדלים בזמן הריצה ב-javascript בין הפתרון הנאיבי לפתרון בתכנון דינמי לחישוב מספר פיבונאצ'י ה43:

```
function recursiveFib(n) {
    if (n <= 1)
        return n;
    return recursiveFib(n-1) + recursiveFib(n-2);
}

function calculateRunningTime(callback) {
    const start = window.performance.now();
     callback(43)
    const end = window.performance.now();
    console.log(`Execution time: ${end - start} ms`);
}
```

calculateRunningTime(recursiveFib)
Execution time: 5246 ms

```
function dynamicFibonacci(n) {
    let a = 0, b = 1, c, i;
    if (n == 0)
        return a;
    for (i = 2; i <= n; i++) {
        c = a + b;
        a = b;
        b = c;
    }
    return b;
}
```

calculateRunningTime(dynamicFibonacci)
Execution time: 0 ms

תרגיל 1 - קבוצה בלתי תלויה בעצים

הגדרה

בהינתן גרף לא מכוון $G = (V\ E)$, קבוצת צמתים $U \subseteq V$ תקרא <u>קבוצה בלתי תלויה</u> אם לכל $u, v \in U$ מתקיים $(u, v) \notin E$.

הציעו אלגוריתם אשר בהינתן עץ לא מכוון $T = (V, E)$ מוצא את הגודל המקסימלי של קבוצה בלתי תלויה בעץ T.

תרגיל 1 - קבוצה בלתי תלויה בעצים

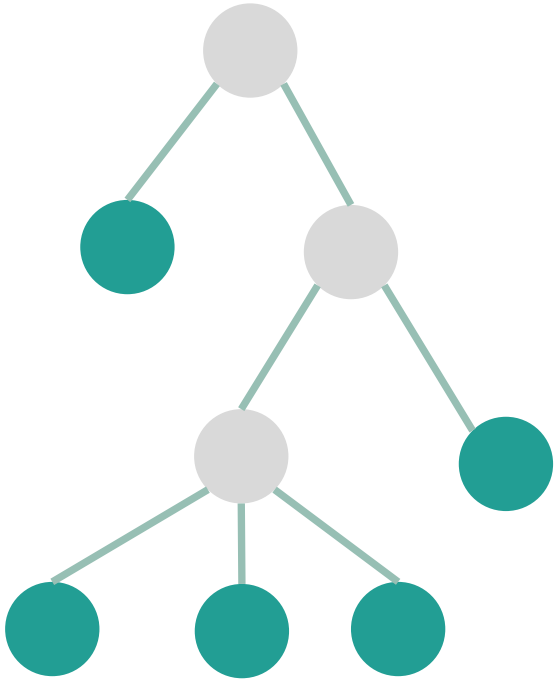
הגדרה

בהינתן גרף לא מכוון $G = (V\ E)$, קבוצת צמתים $U \subseteq V$ תקרא <u>קבוצה בלתי תלויה</u> אם לכל $u, v \in U$ מתקיים $(u, v) \notin E$.

הציעו אלגוריתם אשר בהינתן עץ לא מכוון $T = (V, E)$ מוצא את הגודל המקסימלי של קבוצה בלתי תלויה בעץ T.

אבחנה: אם V נלקח לקבוצה U, אסור לקחת את בניו או אביו בעץ.

דוגמה:



תרגיל 1 - קבוצה בלתי תלויה בעצים

הציעו אלגוריתם אשר בהינתן עץ לא מכוון $T = (V, E)$ מוצא את הגודל המקסימלי של קבוצה בלתי תלויה בעץ T.

ניסוח נוסחה רקורסיבית לתיאור הפתרון

מציאת אלגוריתם "מלמטה למעלה"

תיאור מבנה נתונים השומר את המידע המחושב בכל איטרציה

תרגיל 1 - קבוצה בלתי תלויה בעצים

הציעו אלגוריתם אשר בהינתן עץ לא מכוון $T = (V, E)$ מוצא את הגודל המקסימלי של קבוצה בלתי תלויה בעץ T.

ניסוח נוסחה רקורסיבית לתיאור הפתרון

מציאת אלגוריתם "מלמטה למעלה"

נסמן:

S(v)

גודל הקבוצה הבלתי תלויה המקסימלית בתת העץ ש-v שורשו.

$S^+(V)$

גודל הקבוצה הבלתי תלויה המקסימלית בתת העץ ש-v שורשו **הכוללת את v**.

$S^-(V)$

גודל הקבוצה הבלתי תלויה המקסימלית בתת העץ ש-v שורשו **שאינה כוללת את v**.

$$S(v) = \max\{S^+(V) , S^-(V)\}$$

$$S^+(V) = 1 + \sum_{u \in child(v)} S^- (u)$$

$$S^-(V) = \sum_{u \in child(v)} S(u)$$

תיאור מבנה נתונים השומר את המידע המחושב בכל איטרציה

תרגיל 1 - קבוצה בלתי תלויה בעצים

הציעו אלגוריתם אשר בהינתן עץ לא מכוון $T = (V, E)$ מוצא את הגודל המקסימלי של קבוצה בלתי תלויה בעץ T.

ניסוח נוסחה רקורסיבית לתיאור הפתרון

מציאת אלגוריתם "מלמטה למעלה"

נסמן:

S(v)

גודל הקבוצה הבלתי תלויה המקסימלית בתת העץ ש-v שורשו.

$S^+(V)$

גודל הקבוצה הבלתי תלויה המקסימלית בתת העץ ש-v שורשו **הכוללת את v**.

$S^-(V)$

גודל הקבוצה הבלתי תלויה המקסימלית בתת העץ ש-v שורשו **שאינה כוללת את v**.

$$S(v) = \max\{S^+(V) , S^-(V)\}$$

$$S^+(V) = 1 + \sum_{u \in child(v)} S^-(u)$$

$$S^-(V) = \sum_{u \in child(v)} S(u)$$

תיאור מבנה נתונים השומר את המידע המחושב בכל איטרציה

נחשב לכל צומת v את הערכים $S(v), S^-(V), S^+(V)$ שיאותחלו כמערך אפסים ויחושבו רק פעם אחת.
נשתמש בחישוב שעשינו באיטרציות הבאות במקום לחשב מחדש.

תרגיל 1 - קבוצה בלתי תלויה בעצים

הציעו אלגוריתם אשר בהינתן עץ לא מכוון T = (V, E) מוצא את הגודל המקסימלי של קבוצה בלתי תלויה בעץ T.

ניסוח נוסחה רקורסיבית לתיאור הפתרון

נסמן:

S(v)

גודל הקבוצה הבלתי תלויה המקסימלית בתת העץ ש-v שורשו.

S^+(V)

גודל הקבוצה הבלתי תלויה המקסימלית בתת העץ ש-v שורשו **<u>הכוללת את v</u>**.

S^-(V)

גודל הקבוצה הבלתי תלויה המקסימלית בתת העץ ש-v שורשו **<u>שאינה כוללת את v</u>**.

$$S(v) = \max\{S^+(V) , S^-(V)\}$$

$$S^+(V) = 1 + \sum_{u \in child(v)} S^-(u)$$

$$S^-(V) = \sum_{u \in child(v)} S(u)$$

מציאת אלגוריתם "מלמטה למעלה"

<u>Longest-independent-tree-set</u>(root, S, S^-, S^+):

```
1.  if root = NIL
2.     then return 0
3.  if S[root] != 0
4.     then return
5.  S⁻[root] ← 0
6.  S⁺[root] ← 1
7.  for u ∈ child(root)
8.     Longest-independent-tree-set(u, S, S⁻, S⁺)
9.     do S⁻[root] ← S⁻[root] + S[u]
10.       S⁺[root] ← S⁺[root] + S⁻ [u]
11. S[root] ← max(S⁻[root], S⁺[root] )
12. return S[root]
```

תיאור מבנה נתונים השומר את המידע המחושב בכל איטרציה

נחשב לכל צומת v את הערכים $S^+(V)$,$S^-(V)$,S(v) שיאותחלו כמערך אפסים ויחושבו רק פעם אחת.
נשתמש בחישוב שעשינו באיטרציות הבאות במקום לחשב מחדש.

תרגיל 1 - קבוצה בלתי תלויה בעצים

הציעו אלגוריתם אשר בהינתן עץ לא מכוון T = (V, E) מוצא את הגודל המקסימלי של קבוצה בלתי תלויה בעץ T.

ניסוח נוסחה רקורסיבית לתיאור הפתרון

נסמן:

S(v)
גודל הקבוצה הבלתי תלויה המקסימלית בתת העץ ש-v שורשו.

S^+(V)
גודל הקבוצה הבלתי תלויה המקסימלית בתת העץ ש-v שורשו **<u>הכוללת את v</u>**.

S^-(V)
גודל הקבוצה הבלתי תלויה המקסימלית בתת העץ ש-v שורשו **<u>שאינה כוללת את v</u>**.

$$S(v) = \max\{S^+(V) , S^-(V)\}$$

$$S^+(V) = 1 + \sum_{u \in child(v)} S^-(u)$$

$$S^-(V) = \sum_{u \in child(v)} S(u)$$

מציאת אלגוריתם "מלמטה למעלה"

<u>Longest-independent-tree-set</u>(root, S, S^-, S^+):

```
1.  if root = NIL
2.     then return 0
3.  if S[root] != 0
4.     then return
5.  S⁻[root] ← 0
6.  S⁺[root] ← 1
7.  for u ∈ child(root)
8.     Longest-independent-tree-set(u, S, S⁻, S⁺)
9.     do S⁻[root] ← S⁻[root] + S[u]
10.          S⁺[root] ← S⁺[root] + S⁻ [u]
11. S[root] ← max(S⁻[root], S⁺[root] )
12. return S[root]
```

תיאור מבנה נתונים השומר את המידע המחושב בכל איטרציה

נחשב לכל צומת v את הערכים S(v), S^-(V), S^+(V) שיאותחלו כמערך אפסים ויחושבו רק פעם אחת.
נשתמש בחישוב שעשינו באיטרציות הבאות במקום לחשב מחדש.

זמן ריצה: O(V+E), **מקום**: O(V+E)

תרגיל 2 – בעיית תרמיל הגב בשלמים

נתון תרמיל גב בעל קיבולת משקל W. **נתונים** n עצמים $a_1,a_2,\ldots,a_n$. לכל עצם a_i יש משקל w_i ורווח pi עבור לקיחת המוצר.
- ככל ש-p_i גבוה יותר, חשיבות המוצר עולה.
- כל המשקלים שלמים

מטרה: מצאו אלגוריתם המוצא תת קבוצה $I\in[n]$ <u>אופטימלית</u> של מוצרים כך ש:

1. אין חריגה מקיבול תרמיל הגב $\sum_{i\in I} w_i \leq W$
2. רווח מקסימלי $\max(\sum_{i\in I} p_i)$

תרגיל 2 – בעיית תרמיל הגב בשלמים

נתון תרמיל גב בעל קיבולת משקל W. **נתונים** n עצמים $a_1,a_2,\ldots,a_n$. לכל עצם a_i יש משקל w_i ורווח pi עבור לקיחת המוצר.
- ככל ש-p_i גבוה יותר, חשיבות המוצר עולה.
- כל המשקלים שלמים

מטרה: מצאו אלגוריתם המוצא תת קבוצה $I \in [n]$ <u>אופטימלית</u> של מוצרים כך ש:
1. אין חריגה מקיבול תרמיל הגב $\sum_{i \in I} w_i \leq W$
2. רווח מקסימלי $\max(\sum_{i \in I} p_i)$

דוגמה

W = 50 **I = [a_3, a_2]**

i	1	2	3
w	10	20	30
p	60	100	120

תרגיל 2 – בעיית תרמיל הגב בשלמים

נתון תרמיל גב בעל קיבולת משקל W. **נתונים** n עצמים $a_1,a_2,\ldots,a_n$. לכל עצם a_i יש משקל w_i ורווח pi עבור לקיחת המוצר.
- ככל ש-p_i גבוה יותר, חשיבות המוצר עולה.
- כל המשקלים שלמים

מטרה: מצאו אלגוריתם המוצא תת קבוצה $I \in [n]$ <u>אופטימלית</u> של מוצרים כך ש:
1. אין חריגה מקיבול תרמיל הגב $\sum_{i \in I} w_i \leq W$
2. רווח מקסימלי $\max(\sum_{i \in I} p_i)$

דוגמה

W = 50 **I = [a_3, a_2]**

i	1	2	3
w	10	20	30
p	60	100	120

<u>ניסוח נוסחה רקורסיבית לתיאור הפתרון</u>

תרגיל 2 – בעיית תרמיל הגב בשלמים

נתון תרמיל גב בעל קיבולת משקל W. **נתונים** n עצמים $a_1,a_2,\ldots,a_n$. לכל עצם a_i יש משקל w_i ורווח pi עבור לקיחת המוצר.
- ככל ש-p_i גבוה יותר, חשיבות המוצר עולה.
- כל המשקלים שלמים

מטרה: מצאו אלגוריתם המוצא תת קבוצה $I \in [n]$ אופטימלית של מוצרים כך ש:
1. אין חריגה מקיבול תרמיל הגב $\sum_{i \in I} w_i \leq W$
2. רווח מקסימלי $\max(\sum_{i \in I} p_i)$

דוגמה

W = 50 **I = [a_3, a_2]**

i	1	2	3
w	10	20	30
p	60	100	120

ניסוח נוסחה רקורסיבית לתיאור הפתרון

נסמן ב-F(k,W) את הרווח המקסימלי כאשר בוחרים עצמים מתוך הקבוצה $\{a_1,a_2,\ldots,a_n\}$ בלבד וגודל התרמיל הוא w.
תחת הסימון הזה נרצה לחשב את F(n, W).

$$F(k,w) = \begin{cases} 0 & k = 0 \; or \; W = 0 \\ F(k-1,\mathrm{w}) & w_k > w \\ \max\{F(k-1,w), p_k + F(k-1,w-w_k)\} & w_k \leq w \end{cases}$$

תרגיל 2 – בעיית תרמיל הגב בשלמים

ניסוח נוסחה רקורסיבית לתיאור הפתרון

$$F(k,w)=\begin{cases}0 & k=0 \; or \; W=0\\ F(k-1,\mathrm{w}) & w_k>w\\ \max\{F(k-1,w),p_k+F(k-1,w-w_k)\} & w_k\le w\end{cases}$$

הוכחת הנוסחה: נסמן ב-$OPT_{k,w}$ פתרון אופטימלי שמשקלו F(k,w). נוכיח באמצעות אינדוקציה על k לכל w.

בסיס: עבור k=0, הפתרון האופטימלי הוא 0 לפי הנוסחה.
צעד: נניח נכונות עבור k-1 ולכל w. נוכיח עבור k.
הוכחה: נחלק את הוכחה לשני מקרים:

נשים לב: תמיד מתקיים $F(k-1, w) \le F(k,w)$

תרגיל 2 – בעיית תרמיל הגב בשלמים

ניסוח נוסחה רקורסיבית לתיאור הפתרון

$$F(k,w)=\begin{cases}0 & k=0 \; or \; W=0\\ F(k-1,\mathrm{w}) & w_k > w\\ \max\{F(k-1,w), p_k + F(k-1,w-w_k)\} & w_k \le w\end{cases}$$

הוכחת הנוסחה: נסמן ב-$OPT_{k,w}$ פתרון אופטימלי שמשקלו F(k,w). נוכיח באמצעות אינדוקציה על k לכל w.

בסיס: עבור k=0, הפתרון האופטימלי הוא 0 לפי הנוסחה.
צעד: נניח נכונות עבור k-1 ולכל w. נוכיח עבור k.
הוכחה: נחלק את הוכחה לשני מקרים:

נשים לב: תמיד מתקיים $F(k-1, w) \le F(k,w)$

מקרה א: אם $w_k > w$ אז מתקיים $a_k \notin OPT_{k,w}$ לכן, $F(k,w) = F(k-1, w)$ פתרון אופטימלי לפי הנחת האינדוקציה.

תרגיל 2 – בעיית תרמיל הגב בשלמים

ניסוח נוסחה רקורסיבית לתיאור הפתרון

$$F(k,w)=\begin{cases}0 & k=0 \; or \; W=0\\ F(k-1,\mathrm{w}) & w_k > w\\ \max\{F(k-1,w), p_k + F(k-1,w-w_k)\} & w_k \le w\end{cases}$$

הוכחת הנוסחה: נסמן ב-$OPT_{k,w}$ פתרון אופטימלי שמשקלו F(k,w). נוכיח באמצעות אינדוקציה על k לכל w.

בסיס: עבור k=0, הפתרון האופטימלי הוא 0 לפי הנוסחה.
צעד: נניח נכונות עבור k-1 ולכל w. נוכיח עבור k.
הוכחה: נחלק את הוכחה לשני מקרים:

נשים לב: תמיד מתקיים $F(k-1, w) \le F(k,w)$

מקרה ב: אם $w_k \le w$ ייתכנו שני מצבים:
1. $a_k \notin OPT_{k,w}$ ואז נקבל מקרה זהה ל**מקרה א.**
2. $a_k \in OPT_{k,w}$ – נראה ש- $OPT_{k,w} \setminus \{a_k\}$ קבוצה בעלת רווח $F(k-1, w-w_k)$ במשקל $w-w_k$ המוכלת ב-$\{a_1,a_2,\ldots,a_{k-1}\}$,

כלומר, ערכו של OPT_{kw} (שהוא F(k,w) לפי ההגדרה) מקיים: $F(k,w) \le F(k-1,w-w_k)+p_k$.

נניח בשלילה שמתקיים $F(k,w) < F(k-1,w-w_k)+p_k$ ונסתכל על הקבוצה $OPT_{k-1,w-wk} \cup \{a_k\}$:

- זו קבוצה בעלת משקל w לכל היותר שמוכלת ב-$\{a_1,a_2,\ldots,a_k\}$.

- לפי ההנחה בשלילה, משקלה $F(k-1,w-w_k)+p_k$ יותר גדול מ-F(k,w) בסתירה לאופטימליות של F(k,w).

לכן מתקיים $F(k,w) = F(k-1,w-w_k)+p_k$

מ.ש.ל

תרגיל 2 – בעיית תרמיל הגב בשלמים

ניסוח נוסחה רקורסיבית לתיאור הפתרון

$$F(k,w)=\begin{cases}0 & k=0\ or\ W=0\\ F(k-1,\mathrm{w}) & w_k>w\\ \max\{F(k-1,w),p_k+F(k-1,w-w_k)\} & w_k\le w\end{cases}$$

תיאור מבנה נתונים השומר את המידע

נשמור את חישובי הערך המקסימלי מכל איטרציה במטריצה בגודל (W+1)× (n+1).

מציאת אלגוריתם "מלמטה למעלה"

n - מספר העצמים, **w -** רשימת משקלי העצמים, **p -** רשימת ערכי העצמים, **W-** משקל התרמיל

knapsack(n, w, p, W)

1. **if** n = 0 **or** W = 0
2. **return** 0
3. A ⟵ (n + 1)(W + 1)
4. $A_{0,w}$ ⟵ 0
5. $A_{k,0}$ ⟵ 0
6. **for** k ⟵ 1 **to** n
7. **for** w⟵1 **to** W
8. **if** w[k] <= w
9. **then** $A_{k,w}$⟵max(p[k] + $A_{k-1,w-w[k]}$, $A_{k-1,w}$)
10. **else** A_{kw} ⟵ $A_{k-1,w}$
11. **return** A_{NW}

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	2	2	2	2	2
2	0	2	2	3	3	3
3	0	2	2	3	5	5

תרגיל 2 – בעיית תרמיל הגב בשלמים

ניסוח נוסחה רקורסיבית לתיאור הפתרון

$$F(k,w)=\begin{cases}0 & k=0\ or\ W=0\\ F(k-1,\mathrm{w}) & w_k>w\\ \max\{F(k-1,w),p_k+F(k-1,w-w_k) & w_k\le w\end{cases}$$

תיאור מבנה נתונים השומר את המידע

נשמור את חישובי הערך המקסימלי מכל איטרציה במטריצה בגודל (W+1)× (n+1).

מציאת אלגוריתם "מלמטה למעלה"

n - מספר העצמים, **w -** רשימת משקלי העצמים, **p -** רשימת ערכי העצמים, **W-** משקל התרמיל

knapsack(n, w, p, W)

1. **if** n = 0 **or** W = 0
2. **return** 0
3. A ⟵ (n + 1)(W + 1)
4. $A_{0,w}$ ⟵ 0
5. $A_{k,0}$ ⟵ 0
6. **for** k ⟵ 1 **to** n
7. **for** w⟵1 **to** W
8. **if** w[k] <= w
9. **then** $A_{k,w}$⟵max(p[k] + $A_{k-1,w-w[k]}$, $A_{k-1,w}$)
10. **else** A_{kw} ⟵ $A_{k-1,w}$
11. **return** A_{NW}

עלות: חישוב כל תא במטריצה עולה O(1), סה"כ O(nW).
מקום: O(nW)

נבחין כי סיבוכיות זו **אינה** פולינומית באורך הקלט, שכן היא תלויה בW .

נכונות: נובעת מהוכחת הנוסחה ומכך שאופן המילוי של המטריצה תוכנן כך שבעת חישוב התא $A_{k,w}$ ערכי התאים $A_{k-1,w}$ חושבו לכל w

תרגיל 2 – בעיית תרמיל הגב בשלמים

ניסוח נוסחה רקורסיבית לתיאור הפתרון

$$F(k,w)=\begin{cases}0 & k=0\ or\ W=0\\ F(k-1,\mathrm{w}) & w_k>w\\ \max\{F(k-1,w),p_k+F(k-1,w-w_k)\} & w_k\le w\end{cases}$$

תיאור מבנה נתונים השומר את המידע

נשמור את חישובי הערך המקסימלי מכל איטרציה במטריצה בגודל (W+1)× (n+1).

מציאת אלגוריתם "מלמטה למעלה"

n - מספר העצמים, **w -** רשימת משקלי העצמים, **p -** רשימת ערכי העצמים, **W-** משקל התרמיל

knapsack(n, w, p, W)

1. **if** n = 0 **or** W = 0
2. **return** 0
3. A ⟵ (n + 1)(W + 1)
4. $A_{0,w}$ ⟵ 0
5. $A_{k,0}$ ⟵ 0
6. **for** k ⟵ 1 **to** n
7. **for** w⟵1 **to** W
8. **if** w[k] <= w
9. **then** $A_{k,w}$⟵max(p[k] + $A_{k-1,w-w[k]}$, $A_{k-1,w}$)
10. **else** A_{kw} ⟵ $A_{k-1,w}$
11. **return** A_{NW}

עלות: חישוב כל תא במטריצה עולה O(1), סה"כ O(nW).
מקום: O(nW)

נבחין כי סיבוכיות זו **אינה** פולינומית באורך הקלט, שכן היא תלויה ב W .

נכונות: נובעת מהוכחת הנוסחה ומכך שאופן המילוי של המטריצה תוכנן כך שבעת חישוב התא $A_{k,w}$ ערכי התאים $A_{k-1,w}$ חושבו לכל w

האם אפשר לחסוך <u>במקום</u> עבור חישוב הפתרון האופטימלי?

תרגיל 2 – בעיית תרמיל הגב בשלמים

ניסוח נוסחה רקורסיבית לתיאור הפתרון

$$F(k,w)=\begin{cases}0 & k=0\ or\ W=0\\ F(k-1,\mathrm{w}) & w_k>w\\ \max\{F(k-1,w),p_k+F(k-1,w-w_k)\} & w_k\le w\end{cases}$$

תיאור מבנה נתונים השומר את המידע

נשמור את חישובי הערך המקסימלי מכל איטרציה במטריצה בגודל (W+1)× (n+1).

מציאת אלגוריתם "מלמטה למעלה"

n - מספר העצמים, **w -** רשימת משקלי העצמים, **p -** רשימת ערכי העצמים, **W-** משקל התרמיל

knapsack(n, w, p, W)

1. **if** n = 0 **or** W = 0
2. **return** 0
3. A ⟵ (n + 1)(W + 1)
4. $A_{0,w}$ ⟵ 0
5. $A_{k,0}$ ⟵ 0
6. **for** k ⟵ 1 **to** n
7. **for** w⟵1 **to** W
8. **if** w[k] <= w
9. **then** $A_{k,w}$⟵max(p[k] + $A_{k-1,w-w[k]}$, $A_{k-1,w}$)
10. **else** A_{kw} ⟵ $A_{k-1,w}$
11. **return** A_{NW}

כאשר אין צורך במציאת הפתרון האופטימלי, אלא בערכו בלבד, ניתן לחסוך בסיבוכיות הזיכרון. נשים לב שבכדי לעדכן את השורה הk, אנו נדרשים אך ורק למידע המצוי בשורה k-1.

נוכל להחזיק במקום מטריצה בגודל O(nW), מטריצה בגודל O(W) בלבד

תרגיל 2 – בעיית תרמיל הגב בשלמים

ניסוח נוסחה רקורסיבית לתיאור הפתרון

$$F(k,w)=\begin{cases}0 & k=0 \; or \; W=0\\ F(k-1,\mathrm{w}) & w_k > w\\ \max\{F(k-1,w),p_k+F(k-1,w-w_k)\} & w_k \le w\end{cases}$$

תיאור מבנה נתונים השומר את המידע

נשמור את חישובי הערך המקסימלי מכל איטרציה במטריצה בגודל (W+1)× (n+1).

דוגמה

W = 50 **I = [a_1, a_3]**

i	1	2	3
w	1	2	3
p	2	1	3

מציאת אלגוריתם "מלמטה למעלה"

n - מספר העצמים, **w -** רשימת משקלי העצמים, **p -** רשימת ערכי העצמים, **W-** משקל התרמיל

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	2	2	2	2	2
2	0	2	2	3	3	3
3	0	2	2	3	5	5

בניית פתרון אופטימלי מתוך המידע שחושב

בסיום האלגוריתם נתחיל מהכניסה An,W. לפי הגדרת הרקורסיה,
הערך חושב באמצעות אחת מכניסות המטריצה הבאות:

1. אם $a_n \notin OPT_{n,W}$ אז הערך נלקח מהתא $A_{n-1,W}$
2. $a_n \in OPT_{n,W}$ אז הערך נלקח מהתא $A_{n-1,W-wn}$ בתוספת הערך של a_n

נחזור על התהליך באופן רקורסיבי עד שנגיע לתנאי הבסיס של הנוסחה (k=0/w=0)